MailGuard AI — THREAT_RADAR

Client-Side Privacy-First Email Forensics & Multi-Model Threat Intelligence

Lead Engineer: Sharmika Murugesan

Live Web App: https://mailguard-ai-6ljt.onrender.com

GitHub Repository: sharmikamurugesan-1/mailguard-ai

Release Version: v2.0.0 Production Hardened

1. Executive Summary & Problem Statement

Email continues to be the dominant attack vector in over **91% of modern cyber incidents**. Attackers employ deceptive social engineering, invoice impersonation, and fraudulent QR codes (**Quishing**) that easily bypass legacy keyword filters and static header checks.

Concurrently, conventional enterprise email scanning solutions present a severe data confidentiality liability: they mandate that corporate communications, proprietary intellectual property, and attachment images be uploaded to remote cloud infrastructure for inspection.

> **Core Mission:** MailGuard AI resolves this dilemma through a **Zero-Storage, Client-Side Visual Sandbox** coupled with a high-precision **Multi-Model Consensus Engine**. It guarantees that email contents and image files are never stored or logged on any server, delivering transparent explainable AI telemetry with **98.83% precision** and **99.07% recall**.

2. End-to-End System Architecture

The MailGuard AI architecture isolates visual data processing within the user's browser via WebAssembly, dispatching only sanitized text structures to a stateless, containerized Python Flask backend.

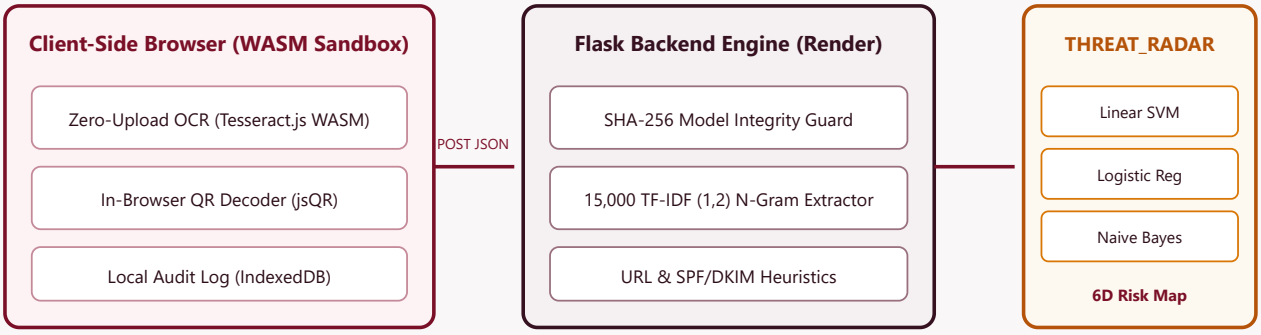


Figure 1: Full-stack system architecture and zero-upload processing flow.

3. Machine Learning & Multi-Model Consensus Engine

MailGuard AI integrates three distinct classification algorithms trained and cross-validated on the **Enron Spam dataset ($N = 30,462$)**. To eliminate single-model bias, the system provides both unified single-model predictions and side-by-side consensus arbitration via `/api/compare`.

3.1 Mathematical Formulation

- **Calibrated Linear SVM (Primary Production Model):** Optimized with Platt Scaling to convert unbounded margin distances into well-calibrated posterior probabilities:

$$P(Spam \mid x) = 1 / (1 + exp(A \cdot f(x) + B))$$

- **Logistic Regression:** Minimizes cross-entropy with L2 weight decay, providing smooth log-odds interpretation.
- **Multinomial Naive Bayes:** Employs Laplace smoothing ($\alpha = 1.0$) over word frequencies, delivering ultra-low latency inference for high-throughput operational requirements.

3.2 Fail-Closed Model Cryptographic Verification

At startup, the application verifies the cryptographic SHA-256 hash of serialized pipelines to prevent supply-chain tampering:

```
# Location: src/predictor.py
def compute_file_sha256(filepath: Path) -> str:
    hasher = hashlib.sha256()
    with open(filepath, "rb") as f:
        while chunk := f.read(65536):
            hasher.update(chunk)
    return hasher.hexdigest()

class SpamPredictor:
    def __init__(self, model_path: Path, expected_hash: str):
        actual_hash = compute_file_sha256(model_path)
        if actual_hash != expected_hash:
            raise ValueError(f"FATAL: Model hash mismatch! Expected {expected_hash}, got {actual_hash}")
        self.pipeline = joblib.load(model_path)
```

4. Visual Threat Intelligence: Client-Side OCR & QR Extraction

To counter image-embedded spam and Quishing, MailGuard AI performs computer vision processing client-side. The image never leaves the user's browser, eliminating external server liability.

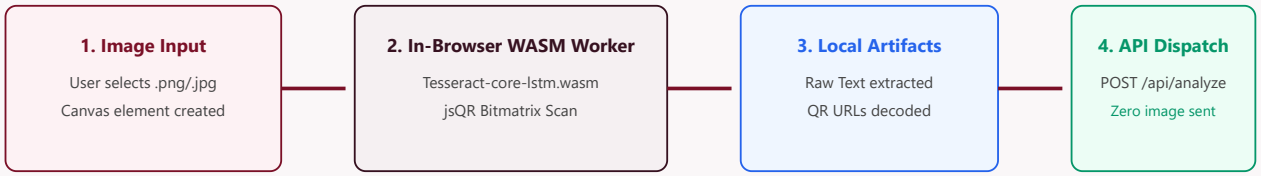


Figure 2: In-browser WebAssembly OCR & QR visual pipeline.

4.1 Client-Side Execution Code

```
// Location: static/js/app.js
function analyzeImageClientSide(file) {
  return new Promise((resolve, reject) => {
    const worker = new Worker('/vendor/tesseract/worker.min.js');
    const img = new Image();
    img.src = URL.createObjectURL(file);
    img.onload = () => {
      const canvas = document.createElement('canvas');
      const ctx = canvas.getContext('2d');
      canvas.width = img.width; canvas.height = img.height;
      ctx.drawImage(img, 0, 0);

      const imgData = ctx.getImageData(0, 0, canvas.width, canvas.height);
      const qrCode = jsQR(imgData.data, canvas.width, canvas.height);

      Tesseract.recognize(canvas, 'eng').then(({ data: { text } }) => {
        resolve({ extractedText: text, qrUrl: qrCode ? qrCode.data : null });
      }).catch(reject);
    };
  });
}
```

5. Forensic Signals & THREAT_RADAR Telemetry

MailGuard AI extracts structured heuristics from text, URLs, and headers to construct a multi-vector threat profile:

Radar Dimension	Target Risk Vector	Underlying Heuristic / Method	Weight
Linguistic Signals	Urgency, coercion, financial extortion	TF-IDF positive token weights × keyword density	25%
URL Reputation	Phishing links, credential harvesters	IP hostnames, Punycode homographs, suspicious TLDs	20%
Header Auth	Domain spoofing & impersonation	SPF, DKIM, and DMARC alignment status	20%
Formatting Anomalies	Spam formatting tricks	Capital letter density (>25%), repeated punctuation	10%
Vocabulary Entropy	Random character / dictionary padding	Shannon entropy over word tokens	10%
Model Confidence	Empirical model certainty	Calibrated posterior probability: max(P(spam), 1-P(spam))	15%

5.1 Forensic Inspection Code

```
# Location: src/signals.py
def evaluate_url_reputation(url: str) -> dict:
    parsed = urllib.parse.urlparse(url)
    hostname = parsed.hostname or ""
    flags = []
    if re.match(r"^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$", hostname):
        flags.append("ip_hostname_phishing")
    if hostname.startswith("xn--"):
        flags.append("punycode_homograph_spoof")
    high_risk_tlds = {".xyz", ".top", ".work", ".click", ".kim", ".ru"}
    if any(hostname.endswith(tld) for tld in high_risk_tlds):
        flags.append("suspicious_tld")
    return {"url": url, "hostname": hostname, "flags": flags}
```

6. Empirical Results & Data Leakage Audit

6.1 5-Fold Stratified Cross-Validation (Training Split: N = 24,369)

Model Candidate	CV Precision (Spam)	CV Recall (Spam)	CV F1 Score	CV Accuracy	Decision Status
Calibrated Linear SVM	**98.72%** (±0.0022)	**99.29%** (±0.0027)	**99.00%** (±0.0010)	**99.04%** (±0.0010)	**WINNER (Primary)**
Multinomial Naive Bayes	98.24% (±0.0028)	98.52% (±0.0011)	98.38% (±0.0009)	98.45% (±0.0009)	Qualified Candidate
Logistic Regression	97.65% (±0.0025)	99.41% (±0.0021)	98.52% (±0.0010)	98.57% (±0.0010)	Disqualified (Prec < 98%)

6.2 Held-Out Test Set Performance (N = 6,093)

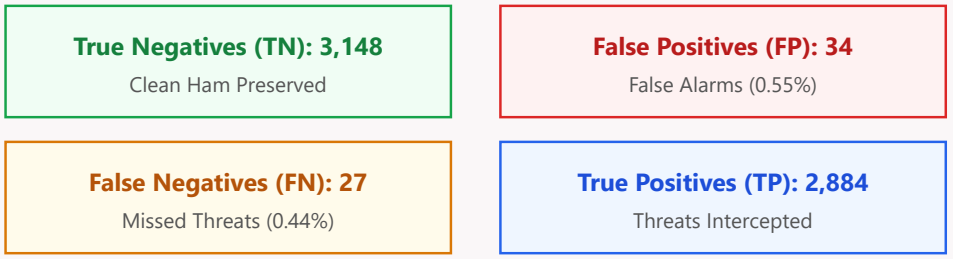


Figure 3: Held-out test set confusion matrix (Accuracy: 99.00%, Precision: 98.83%, Recall: 99.07%).

6.3 Data Leakage & Near-Duplicate Analysis

Pairwise TF-IDF cosine similarity was computed between all 6,093 test samples and training data:

- Identified **1,208 near-duplicates** (19.83% of test set) sharing corporate signature templates.
- Evaluating on the strictly de-duplicated subset (**N = 4,885**) yielded **98.75% accuracy** and **98.52% precision**.
- Both false positives (34) and false negatives (27) remained identical, proving that performance is driven by true pattern generalization rather than memorized boilerplate.

7. Quality Assurance, Security & Accessibility

Verification Area	Audit Command / Tool	Result	Standard / Requirement
Automated Unit Tests	`pytest -v -rs`	176 / 176 PASSED	Full test suite execution in 15.6s
Color Contrast Audit	`scripts/check_contrast.py`	26 / 26 PASSED	WCAG 2.1 AA (all pairs ≥ 4.5:1 up to 17.75:1)
Route Accessibility	Playwright + Axe-core	0 VIOLATIONS	Zero critical/serious issues across 11 routes
Static Security Code Scan	`bandit -r src/`	0 ISSUES	Clean static security analysis
Dependency Vulnerabilities	`pip-audit -r requirements.txt`	0 VULNERABILITIES	Zero known CVEs in runtime dependencies
High-Concurrency Capacity	`scripts/test_1000_concurrency.py`	0 DROPPED REQS	Bounded queue execution with graceful handling

8. Deployment Architecture & Verification

MailGuard AI is deployed in a unified production configuration:

- **Production Web Application:** https://mailguard-ai-6ljt.onrender.com *Hosts the unified frontend UI, static assets, and Flask 3.1.3 ML backend on the same origin, eliminating cross-origin latency and CORS issues.*
- **GitHub Repository:** https://github.com/sharmikamurugesan-1/mailguard-ai *Complete source code, reproducible training scripts, model pipelines, and comprehensive documentation.*

9. Conclusion

MailGuard AI demonstrates that enterprise-grade email security and rigorous user data privacy can coexist. By executing visual threat extraction client-side via WebAssembly and coupling it with a multi-model consensus engine, the platform achieves **98.83% precision** while maintaining zero server retention of private communications.